Three paths to a domain-specific agent harness

Different starting points shape how methodology, implementation, and change are handled

APPROACH 1

Custom agentic application

Domain problem + expertise

↓ implement directly in a framework

Framework constructs

workflows · agents · RAG · tools · state

One domain-specific system

Architecture is expressed in application code

SPEED TO FIRST SYSTEM

Variable; depends on scope and expertise

ADAPTING TO CHANGE

Changes may require code and design updates

APPROACH 2

Harness specialization

Existing general-purpose harness

↓ manually specialize its capabilities

Skills · tools · instructions

possibly subagents and orchestration

Specialized general harness

Design and instantiation are largely manual

SPEED TO FIRST SYSTEM

Can be quick when the base harness fits

ADAPTING TO CHANGE

Depends on modularity and harness constraints

APPROACH 3

HarnessWeaver

Domain + methodology made explicit

↓ synthesize and iteratively refine

Generated, evolvable harness

evaluate · revise · develop further

Prototype before commitment

then retain or refine into a realization

SPEED TO ITERATE

Designed to make prototyping efficient

ADAPTING TO CHANGE

Explicit inputs support faster evolution

HarnessWeaver's position

A prototyping and evolution layer that delays commitment to a concrete agent architecture or runtime.

Qualitative tendencies, not benchmark results. HarnessWeaver does not claim to automatically find an optimal harness.